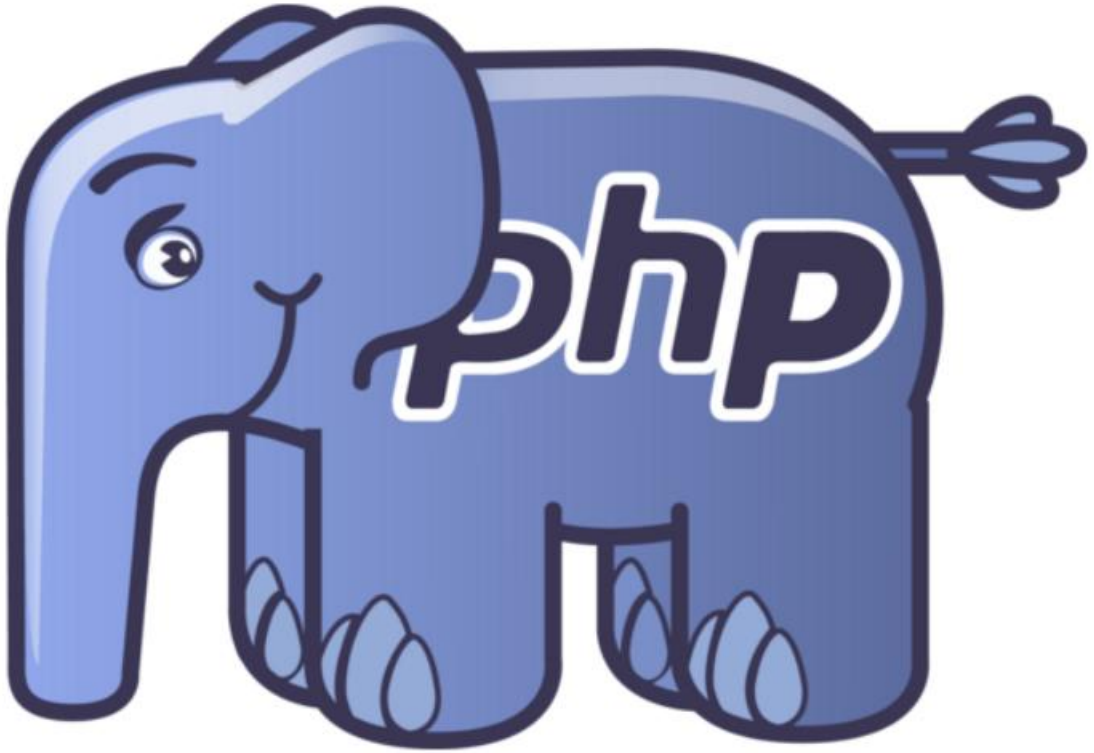
PHP Basics

Server-Side Logic in Integrative Programming

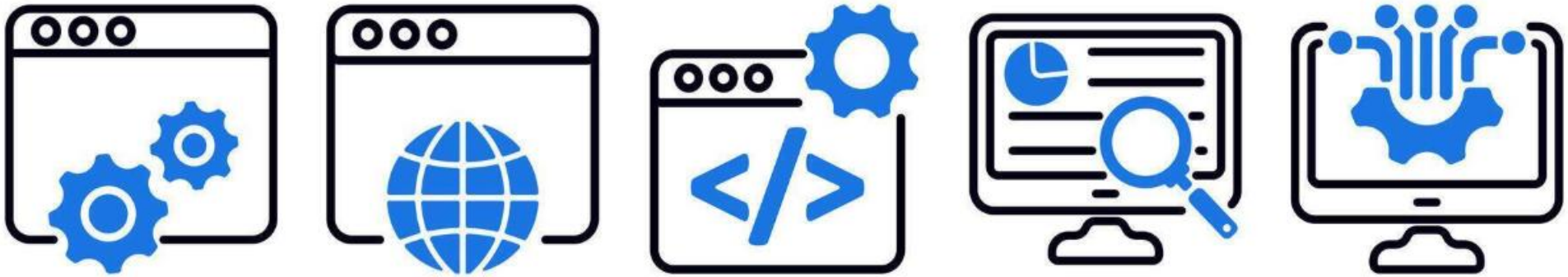
Overview

- Web-Based Programming
- Static vs Dynamic Websites
- PHP Syntax, Variables, Data Types, and Operators
- Control Structures and Loops
- Working with Forms (GET & POST)
- Including and Requiring Files
- Error Handling Basics



Purpose of Web-Based Programming

- What is web-based programming?
  - Web-based programming is the process of creating applications that run on a **web server** and are accessed through a **web browser**.
  - It combines **front-end technologies** (HTML, CSS) with **server-side scripting** (PHP) to deliver interactive and data-driven websites.



Why do we use web-based programming?

- Web-based programming allows developers to:
  - Create websites that respond to user actions
  - Process and store user data
  - Generate content dynamically
  - Connect to databases and servers
  - Make applications accessible anytime, anywhere

Static vs Dynamic Websites

- **Static Website** - Content is fixed. Same content for every user. Created using designer or formatting tools (e.g. HTML and CSS) only. Requires manual updates.
  - **Example:** Personal profile pages, brochure websites

- **Dynamic Website** - Content changes based on user input or data. Uses server-side languages like PHP. Can connect to databases. Automatically updates content.
  - **Example:** Login systems, online enrollment, e-commerce sites

Importance of Dynamic Website

- Dynamic websites are important because they:
  - Enable **user interaction** (forms, logins, comments)
  - Provide **personalized content**
  - Support **real-world applications**
  - Improve **user experience**
  - Form the foundation of **modern web systems**

How PHP Fits In?

- PHP acts as the **logic layer** of a web application:
  - Receives input from users
  - Processes data on the server
  - Generates HTML output
  - Works with databases and APIs

📌 Without PHP, websites remain static. With PHP, websites become intelligent and interactive.

PHP Syntax, Variables, Data Types, and Operators

What is PHP?

- PHP (Hypertext Preprocessor) is a **server-side scripting language** used to create dynamic web pages. PHP code is executed on the server, and the result (usually HTML) is sent to the browser.
- PHP scripts are enclosed within:

```
<?php

// PHP code here

?>
```

PHP Basic Syntax

- A PHP script starts with **<?php** and ends with **?>**
- The default file extension for PHP files is ".php"
- A PHP file normally contains HTML tags, and some PHP scripting code
- PHP statements are terminated by semicolon (;)
- In PHP, all user-defined functions, classes, and keywords (e.g. if, else, while, echo, etc.) are not case-sensitive

Comments in PHP

- // Single line comment (C++ and Java-style comment)
- # Single line comment (Shell-style comments)
- /* Multiple line comment (C-style comments) */

Variables

- Variables start with a **$**
- No need to declare a data type explicitly
- PHP is **loosely typed** – means PHP automatically determines and can change a variable's data type based on its value, without requiring the programmer to declare it.

```
<?php
$value = 10;        // Integer
$value = "Ten";     // Now a String
$value = 10.5;      // Now a Float
?>
```

Common Data Types

Data Type	Example
String	“Hello”
Integer	10
Float	3.1416
Boolean	true, false
Array	[“Apple”, “Banana”]

```
<?php
$price = 99.50;
$subjects = ["Math", "Science", "English"];
?>
```

Operators - Arithmetic

Operator	Name	Example	Result
+	Addition	$x + $y	Sum of $x and $y
-	Subtraction	$x - $y	Difference of $x and $y
*	Multiplication	$x * $y	Product of $x and $y
/	Division	$x / $y	Quotient of $x and $y
%	Modulus	$x % $y	Remainder of $x divided by $y
**	Exponentiation	$x ** $y	Result of raising $x to the $y'th power

```
<?php
$total = 100 + 50;
$discount = $total * 0.10;
?>
```

Operators - Comparison

Operator	Name	Example	Result
==	Equal	$x == $y	Returns true if $x is equal to $y
===	Identical	$x === $y	Returns true if $x is equal to $y, and they are of the same type
!=	Not equal	$x != $y	Returns true if $x is not equal to $y
<>	Not equal	$x <> $y	Returns true if $x is not equal to $y
!==	Not identical	$x !== $y	Returns true if $x is not equal to $y, or they are not of the same type
>	Greater than	$x > $y	Returns true if $x is greater than $y
<	Less than	$x < $y	Returns true if $x is less than $y
>=	Greater than or equal to	$x >= $y	Returns true if $x is greater than or equal to $y
<=	Less than or equal to	$x <= $y	Returns true if $x is less than or equal to $y
<=>	Spaceship	$x <=> $y	Returns an integer less than, equal to, or greater than zero, depending on if $x is less than, equal to, or greater than $y. Introduced in PHP 7.

Operators - Logical

Operator	Name	Example	Result
and	And	$x and $y	True if both $x and $y are true
or	Or	$x or $y	True if either $x or $y is true
xor	Xor	$x xor $y	True if either $x or $y is true, but not both
&&	And	$x && $y	True if both $x and $y are true
||	Or	$x || $y	True if either $x or $y is true
!	Not	!$x	True if $x is not true

Operators - String

Operator	Name	Example	Result
.	Concatenation	$txt1 . $txt2	Concatenation of $txt1 and $txt2
.=	Concatenation assignment	$txt1 .= $txt2	Appends $txt2 to $txt1

PHP Control Structures and Loops

Conditional Statements

Control structures allow PHP programs to make decisions and repeat actions, enabling dynamic and responsive behavior.

- *if* statement - executes some code if one condition is true
- *if...else* statement - executes some code if a condition is true and another code if that condition is false
- *if...elseif...else* statement - executes different codes for more than two conditions
- *switch* statement - selects one of many blocks of code to be executed

```php
<?php
$grade = 85;

if ($grade >= 75) {
    echo "Passed";
} else {
    echo "Failed";
}
?>
```

```php
<?php
$day = "Monday";

switch ($day) {
    case "Monday":
        echo "Start of the week";
        break;
    case "Friday":
        echo "Almost weekend";
        break;
    default:
        echo "Regular day";
}
?>
```

Loops

PHP loops are used to execute the same block of code again and again, as long as a certain condition is true.

for loop

```
<?php
for ($i = 1; $i <= 5; $i++) {
    echo "Number: $i <br>";
}
?>
```

while loop

```
<?php
$count = 1;
while ($count <= 3) {
    echo "Count: $count <br>";
    $count++;
}
?>
```

foreach loop

```
<?php
$colors = ["Red", "Green", "Blue"];

foreach ($colors as $color) {
    echo "$color <br>";
}
?>
```

Applications

- Control structures and loops are used in:
  - Displaying lists of data
  - Processing multiple records
  - Repeating form validation checks
  - Generating dynamic HTML content

Working with Forms (GET and POST)

Passing Information to the Server

- Sometimes, we require additional values be passed from client to server
  - Login: username and password
  - Form information to be stored on server
- Using superglobal variables, it allows the passing of information from the client to the server.
- 2 components to a form:
  - The HTML generating the form itself
  - The server-side script that the form data is sent to (via GET or POST), taking care of the processing involved
    - Server should respond appropriately, redirecting the user to the appropriate destination or generating the appropriate page

GET and POST Superglobals

- Scripts will interact with their clients using one of the two HTTP methods. The methods are GET and POST
- When a form is submitted using the GET method, its values are encoded directly in the query string portion of the URL
  - Example : http://www.yourdomain.com/yourpage.php?firstparam=firstvalue&secondparam=secondvalue
- When a form is submitted using the POST method, its values will not be displayed the query string portion of the URL
- Forms are generally used to collect data, whether the data needs to be stored on the server (registration) or checked against the server (login)

Forms: Client Side

```
<html>
<head><title> A Form Example </title></head>
<body>
  <form action="welcome.php" method="post">
    Name: <br /> <input type="text" name="name" />><br />
    Phone Number: <br /> <input type="text" name="phone" />><br/>
    <input type="submit" value="Submit">
  </form>
</body>
</html>
```

- form action – where to send the form data
- method – how to send the data (GET or POST)
- Name attributes become the keys used to access the corresponding fields in the $_GET or $_POST arrays

Forms: PHP Processing

```
<html>
<head><title>This is welcome.php</title></head>
<body>
 The name that was submitted was: <?php echo $_POST['name']; ?><br />
 The phone number that was submitted was: <?php echo $_POST['phone'];
?>
</body>
</html>
```

- A simple PHP file that displays what was entered into the form
- Can do many other things server-side depending on the situation
- Note the use of $_POST[]

Input Validation Example

```php
<?php
if (empty($_POST['password'])) {
    echo "Password is required";
}
?>
```

- Input validation ensures that the data received from users is correct, complete, and safe before it is processed.
- This validation is necessary because:
  - Users may submit empty or incomplete data
  - Forms can be submitted without user interaction (e.g., page refresh, direct URL access)
  - Invalid input can cause logic errors or system crashes
  - Proper validation improves security and user experience
- More on these on the next modules.

Including and Requiring Files

Include and Require

- PHP allows code **reuse** through **include** and **require**, helping maintain clean and modular applications.

Command	Description	Example

include	Warning if file missing
	Script continues
		`<?php`
		`include "header.php";`
		`echo "Welcome to the homepage";`
		`?>`

require	Fatal error if file missing
	Script stops
		`<?php`
		`require "config.php";`
		`?>`

- Applications:
  - Reusable headers and footers
  - Centralized database connections
  - Clean project structure
  - Easier maintenance

Error Handling Basics

Error Handling Basics

- Error handling helps developers **detect, display, and manage errors** during program execution, improving reliability and debugging.

Displaying Errors (Development Only)

```
<?php
   error_reporting (E_ALL ^ E_WARNING)
?>
```

- error_reporting() defines which types of errors PHP should report
- E_ALL tells PHP to report all errors, including: Notices, Warnings Deprecated, features, Fatal errors
- ^ symbol means to exclude

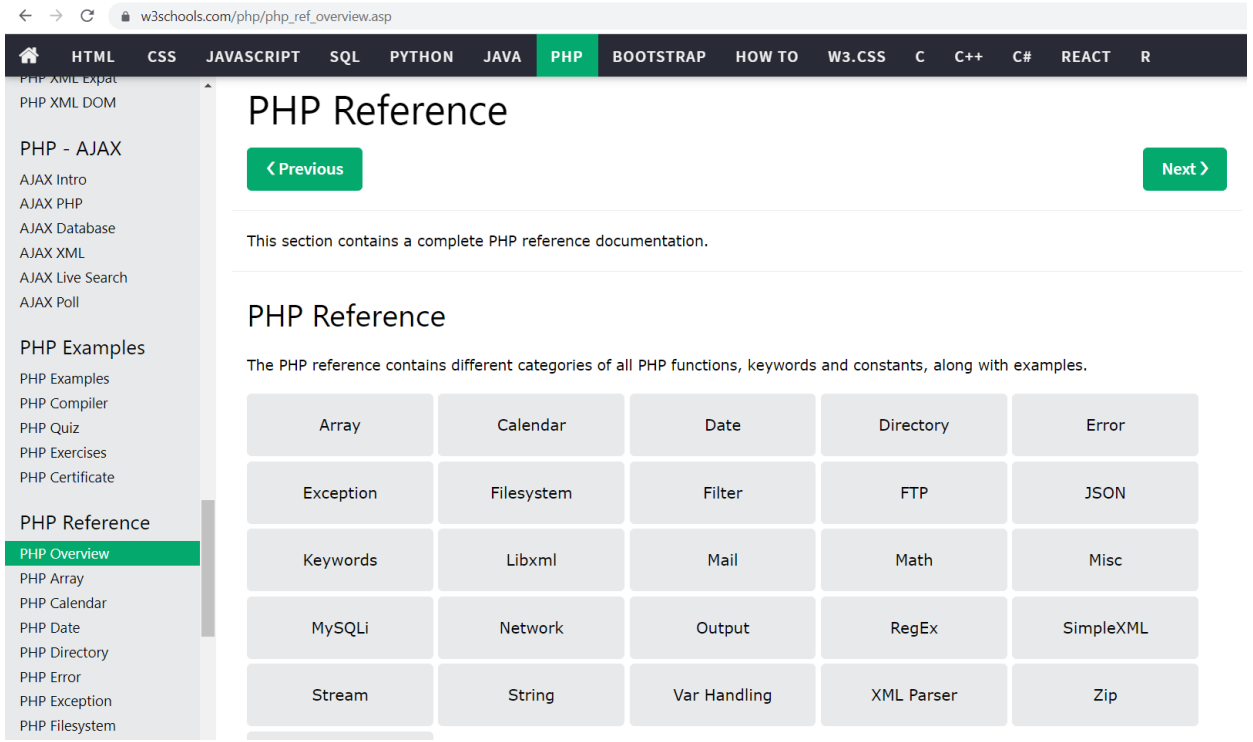
Basic Error Handling with *if*

```
<?php
$file = "data.txt";
if (!file_exists($file)) {
    echo "File not found";
}
?>
```

Use PHP Variable Handling Functions:

- is_null()
- isset()
- empty()

and a lot more...



PHP Basics

Server-Side Logic in Integrative Programming